

SENAI AFONSO GRECO
CURSO: DESENVOLVIMENTO DE SISTEMAS

DANILO DA SILVA SANTOS

**RELATÓRIO DO DESENVOLVIMENTO DO SISTEMA
RAWG GAME EXPLORER**

CIDADE
2026

DANILO DA SILVA SANTOS

**RELATÓRIO DO DESENVOLVIMENTO DO SISTEMA
RAWG GAME EXPLORER**

Relatório apresentado como requisito avaliativo da disciplina de desenvolvimento de sistemas, com o objetivo de documentar o processo de criação de uma aplicação desktop desenvolvida em C# com WPF, integração com API externa, banco de dados local SQLite e aplicação do padrão MVVM.

Professor(a): Frederico Martins Aguiar

CIDADE
2026

RESUMO

O presente relatório apresenta o desenvolvimento do sistema RAWG Game Explorer, uma aplicação desktop criada em C# com WPF, utilizando .NET 8, Entity Framework Core, SQLite e consumo de APIs externas. O sistema tem como objetivo permitir a busca de jogos por meio da API RAWG, exibir os resultados em uma interface gráfica, salvar jogos selecionados em um banco de dados local, gerenciar esses registros e enviá-los para uma API externa. Durante o desenvolvimento, foram aplicados conceitos de organização em camadas, padrão MVVM, persistência de dados, consumo de API com HttpClient e construção de interfaces em XAML. Também foram realizadas correções e melhorias relacionadas à estrutura do banco de dados, tratamento de campos nulos, validação de dados, ajustes no envio para API e aprimoramento visual das telas. Como resultado, obteve-se uma aplicação funcional, organizada e capaz de realizar o fluxo completo de busca, salvamento, gerenciamento local e envio externo de dados.

Palavras-chave: C#. WPF. SQLite. API. MVVM. RAWG.

SUMÁRIO

1 INTRODUÇÃO	5
2 OBJETIVOS	5
2.1 Objetivo geral	5
2.2 Objetivos específicos	5
3 TECNOLOGIAS UTILIZADAS	6
4 ESTRUTURA DO PROJETO	6
5 DESENVOLVIMENTO DO SISTEMA	7
5.1 Tela principal	7
5.2 Busca de jogos na API RAWG	7
5.3 Salvamento no banco SQLite	7
5.4 Tela Meu Banco de Dados Local	8
5.5 Atualização e exclusão de registros	8
5.6 Envio para API externa	8
6 DIFICULDADES ENCONTRADAS	8
7 MUDANÇAS REALIZADAS	9
8 INSTRUÇÕES PARA EXECUÇÃO	10
9 CONSIDERAÇÕES FINAIS	10

1 INTRODUÇÃO

O desenvolvimento de aplicações desktop ainda possui grande importância em sistemas que necessitam de uma interface gráfica local, controle de dados e integração com serviços externos. Nesse contexto, foi desenvolvido o sistema RAWG Game Explorer, uma aplicação em C# com WPF que permite consultar jogos em uma API externa, salvar informações localmente e posteriormente enviar esses dados para outra API.

O sistema foi criado com o objetivo de aplicar conceitos de programação orientada a objetos, consumo de APIs, persistência de dados com SQLite, organização por camadas e uso do padrão MVVM. A aplicação possui duas telas principais: a tela inicial, utilizada para pesquisar jogos na API RAWG, e a tela de banco local, utilizada para visualizar, atualizar, excluir e enviar os jogos salvos.

Durante o desenvolvimento, foram encontradas dificuldades relacionadas à estrutura do banco de dados, ao tratamento de campos nulos, ao envio de dados para API externa e à organização da interface em XAML. Essas dificuldades foram corrigidas ao longo do processo, resultando em uma aplicação funcional e mais organizada.

2 OBJETIVOS

2.1 Objetivo geral

Desenvolver uma aplicação desktop em C# com WPF capaz de pesquisar jogos na API RAWG, salvar registros em banco local SQLite, gerenciar os dados salvos e enviá-los para uma API externa.

2.2 Objetivos específicos

Os objetivos específicos do projeto foram:

- Criar uma interface gráfica utilizando WPF e XAML;
- Consumir dados da API RAWG por meio de requisições HTTP;
- Exibir os jogos pesquisados em uma tabela;
- Salvar o jogo selecionado em um banco de dados SQLite;
- Criar uma tela separada para gerenciamento do banco local;
- Permitir atualização e exclusão de registros salvos;
- Implementar exclusão de múltiplos registros marcados;
- Enviar os jogos salvos para uma API externa;
- Aplicar o padrão MVVM para organizar a lógica do sistema;
- Melhorar a aparência visual da aplicação.

3 TECNOLOGIAS UTILIZADAS

Para o desenvolvimento do sistema foram utilizadas as seguintes tecnologias:

- C#, como linguagem principal de programação;
- WPF, para construção da interface gráfica;
- XAML, para estruturação visual das telas;
- .NET 8, como plataforma de execução;
- Entity Framework Core, para comunicação com o banco de dados;
- SQLite, como banco de dados local;
- HttpClient, para consumo de APIs externas;
- API RAWG, para consulta de jogos;
- API externa, para envio dos jogos salvos;
- Padrão MVVM, para separação entre interface e lógica de negócio.

4 ESTRUTURA DO PROJETO

O projeto foi organizado em pastas, separando as responsabilidades de cada parte da aplicação. A estrutura principal ficou da seguinte forma:

```

RawgApi
■
■■■■ Data
■   ■■■ LocalDbContext.cs
■
■■■■ Models
■   ■■■ Games.cs
■   ■■■ RawgResponse.cs
■   ■■■ RelayCommand.cs
■
■■■■ Services
■   ■■■ RawgApiService.cs
■   ■■■ Aluno2ApiService.cs
■
■■■■ ViewModels
■   ■■■ MainViewModel.cs
■   ■■■ BancoLocalViewModel.cs
■
■■■■ Views
■   ■■■ MainWindow.xaml
■   ■■■ MainWindow.xaml.cs
■   ■■■ BancoLocalWindow.xaml
■   ■■■ BancoLocalWindow.xaml.cs
■
■■■■ App.xaml
■■■■ localdb.db
  
```

A pasta Data contém a classe responsável pela conexão com o banco SQLite. A pasta Models armazena as classes de modelo, como a classe Games. A pasta Services

concentra as classes responsáveis pela comunicação com APIs externas. A pasta ViewModels contém a lógica das telas, seguindo o padrão MVVM. Por fim, a pasta Views contém as telas da aplicação.

5 DESENVOLVIMENTO DO SISTEMA

5.1 Tela principal

A tela principal do sistema é responsável pela busca dos jogos na API RAWG. Nela, o usuário pode digitar o nome de um jogo, realizar a busca, visualizar os resultados e salvar o jogo selecionado no banco local.

A tela principal contém campo de pesquisa, botão Buscar RAWG, botão Salvar Selecionado, botão Meu Banco Local, tabela com os resultados da busca e botão X para fechar o programa.

Uma alteração importante realizada foi deixar a tela principal iniciando limpa. Antes, a aplicação carregava automaticamente os jogos salvos no banco local ao iniciar, o que causava confusão. Após a correção, os jogos aparecem apenas quando o usuário realiza uma busca.

5.2 Busca de jogos na API RAWG

A busca de jogos foi implementada por meio da classe RawgApiService, utilizando HttpClient. O usuário informa um termo de pesquisa, e o sistema envia uma requisição para a API RAWG.

Os dados retornados são tratados e convertidos para objetos da classe Games. Entre as informações exibidas estão capa do jogo, ID original da RAWG, nome, descrição, avaliação, Metacritic e data de upload.

Também foi realizado tratamento para evitar erros quando algum dado retornado pela API viesse nulo, como a imagem do jogo.

5.3 Salvamento no banco SQLite

O banco de dados local utilizado foi o SQLite, com acesso realizado por meio do Entity Framework Core. A classe LocalDbContext é responsável pela configuração do banco e pela tabela Games.

Inicialmente, o sistema salvava todos os jogos retornados pela busca. Posteriormente, essa lógica foi corrigida para salvar apenas o jogo selecionado pelo usuário.

O fluxo correto passou a ser: pesquisar um jogo, selecionar uma linha na tabela, clicar em Salvar Selecionado e salvar o jogo no banco local SQLite. Também foi implementada uma verificação para evitar registros duplicados.

5.4 Tela Meu Banco de Dados Local

Foi criada uma segunda tela chamada Meu Banco de Dados Local, com a finalidade de separar a busca externa do gerenciamento dos dados locais.

Essa tela possui os botões Carregar Banco, Atualizar, Excluir, Selecionar Todos, Excluir Marcados e Enviar API. Assim como a tela principal, a tela do banco local também foi ajustada para iniciar limpa. Os dados só aparecem quando o usuário clica em Carregar Banco.

5.5 Atualização e exclusão de registros

Na tela do banco local, o usuário pode editar informações diretamente na tabela e clicar em Atualizar. O sistema busca o jogo no banco pelo ID original da RAWG e atualiza os campos editados.

Também foi implementada a exclusão de registros. O usuário pode excluir um único jogo selecionado ou marcar vários jogos e utilizar a opção Excluir Marcados. Antes da exclusão, o sistema exibe uma mensagem de confirmação.

5.6 Envio para API externa

O envio de dados para a API externa foi mantido somente na tela Meu Banco de Dados Local, pois o objetivo é enviar apenas os jogos já salvos no banco SQLite.

Durante essa etapa, foram necessárias conversões para adequar os dados ao formato esperado pela API. O ID do jogo precisou ser enviado como texto, a classificação como número inteiro e a data de upload em formato UTC.

```
Id = jogo.Id.ToString()
Classificacao = ConverterClassificacao(jogo.Classificacao)
Upload = ConverterParaUtc(jogo.Upload)
```

Após esses ajustes, o envio para a API passou a funcionar corretamente.

6 DIFICULDADES ENCONTRADAS

Durante o desenvolvimento do sistema, foram encontradas diversas dificuldades técnicas.

A primeira dificuldade ocorreu com o banco SQLite. O sistema apresentou o erro: SQLite Error 1: table Games has no column named Classificacao. Esse erro aconteceu porque o banco já havia sido criado antes da inclusão da propriedade Classificacao na classe Games. A solução foi recriar o banco local, apagando o arquivo antigo localdb.db.

Outra dificuldade foi o erro de campo obrigatório nulo: NOT NULL constraint failed: Games.ImagemUrl. Esse problema ocorreu porque alguns jogos retornados pela API RAWG não possuíam imagem. Para resolver, foi feita a normalização dos dados antes do

salvamento, substituindo valores nulos por `string.Empty` ou `"0"`.

Também houve dificuldade com o salvamento dos dados. Inicialmente, o sistema salvava a lista inteira de jogos retornados pela API. Esse comportamento foi ajustado para salvar somente o jogo selecionado.

No desenvolvimento das telas, ocorreram erros de XAML, como tags `Grid` e `Window` não fechadas. Esses erros foram corrigidos com a reestruturação dos arquivos XAML completos.

A integração com a API externa também apresentou dificuldades. Primeiro, a URL estava com aspas extras, tornando o endereço inválido. Depois, a API retornou erros de validação, exigindo que o ID fosse enviado como string, que a classificação fosse enviada como inteiro e que a data estivesse em UTC. Após as conversões necessárias, a comunicação com a API funcionou.

7 MUDANÇAS REALIZADAS

As principais mudanças realizadas no sistema foram:

- Criação da busca de jogos na API RAWG;
- Criação do banco local SQLite;
- Criação da model Games;
- Criação do serviço RawgApiService;
- Criação do serviço Aluno2ApiService;
- Criação da tela principal;
- Criação da tela Meu Banco de Dados Local;
- Implementação do salvamento de jogo selecionado;
- Implementação do carregamento dos dados salvos;
- Implementação da atualização de registros;
- Implementação da exclusão de registros;
- Implementação da exclusão de vários registros marcados;
- Implementação do envio para API externa;
- Correção da URL da API externa;
- Correção do formato do JSON enviado para API;
- Correção do ID enviado como string;
- Correção da classificação enviada como inteiro;
- Correção da data de upload enviada em UTC;
- Remoção do botão de envio da tela principal;
- Adição do botão de envio na tela do banco local;
- Adição do botão X para fechar as janelas;
- Ajuste para a tela principal iniciar limpa;

- Ajuste para a tela do banco local iniciar limpa;
- Adição do ID local sequencial;
- Ajustes visuais no DataGrid;
- Remoção das faixas brancas da tabela;
- Melhoria na visualização das imagens, colunas e datas.

8 INSTRUÇÕES PARA EXECUÇÃO

Para executar o projeto, é necessário ter instalado Visual Studio 2022 ou superior, .NET 8 SDK, pacotes NuGet do Entity Framework Core, pacote SQLite para Entity Framework Core e conexão com a internet.

Os pacotes necessários são:

```
Microsoft.EntityFrameworkCore
Microsoft.EntityFrameworkCore.Sqlite
Microsoft.EntityFrameworkCore.Tools
```

Também é possível instalar pelo terminal:

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

No arquivo RawgApiService.cs, deve ser configurada a chave da API RAWG:

```
private readonly string _apiKey = "SUA_CHAVE_RAWG_AQUI";
```

No arquivo Aluno2ApiService.cs, deve ser conferida a URL da API externa:

```
private readonly string _aluno2Url = "https://api-rawg.runasp.net/api/Jogos";
```

Para executar pelo Visual Studio, deve-se abrir a solução, restaurar os pacotes NuGet, confirmar a chave da API RAWG, confirmar a URL da API externa, recompilar a solução e executar o projeto com F5.

Para executar pelo terminal, utilize os comandos:

```
dotnet restore
dotnet build
dotnet run
```

9 CONSIDERAÇÕES FINAIS

O desenvolvimento do sistema RAWG Game Explorer permitiu aplicar conceitos importantes de programação desktop, consumo de APIs, banco de dados local e integração entre sistemas.

O projeto possibilitou a utilização do padrão MVVM, a criação de interfaces em WPF, o uso do Entity Framework Core com SQLite e a comunicação com APIs externas por meio do HttpClient.

Apesar das dificuldades encontradas, como erros no banco SQLite, problemas com valores nulos, falhas de validação na API externa e erros de estrutura no XAML, todas as etapas foram corrigidas. O resultado final foi uma aplicação funcional, organizada e capaz de realizar o fluxo completo de busca, salvamento, gerenciamento local e envio de jogos para uma API externa.